
```
lowererrors = [abs(lowhappy - meanlower) for lowhappy in low_social_happiness]
highererrors = [abs(highhappy - meanhigher) for highhappy in
high_social_happiness]

total_error = sum(lowererrors) + sum(highererrors)
```

This code takes the sum of all prediction errors for all respondents. It defines `lowererrors`, a list containing the prediction error for each less social respondent, and `highererrors`, a list containing the prediction error for each more social respondent. Notice that we took the absolute value so that we're adding only non-negative numbers to calculate the error sum. When we run this code, we find that our total error is about 60224. This number is much higher than zero, but if you consider that this is a sum of errors for more than 40,000 respondents whose happiness we predicted using a tree with only two branches, suddenly it doesn't seem so bad.

We can try different split points to see if our error improves. For example, we can classify everyone with a social rating higher than 4 as high social and everyone with a social rating of 4 or lower as low social, and compare the resulting error rates. Or we could use 6 as our split point instead. In order to get the highest possible accuracy, we should check every possible split point in order, and choose the split point that leads to the lowest possible error. [Listing 9-2](#) contains a function that accomplishes this.

```
def get_splitpoint(allvalues, predictedvalues):
    lowest_error = float('inf')
    best_split = None
    best_lowermean = np.mean(predictedvalues)
    best_highermean = np.mean(predictedvalues)
    for pct1 in range(0,100):
        split_candidate = np.percentile(allvalues, pct1)

        loweroutcomes = [outcome for value, outcome in
zip(allvalues, predictedvalues) if \value <= split_candidate]
        higheroutcomes = [outcome for value, outcome in
zip(allvalues, predictedvalues) if \value > split_candidate]

        if np.min([len(loweroutcomes), len(higheroutcomes)]) > 0:
            meanlower = np.mean(loweroutcomes)
            meanhigher = np.mean(higheroutcomes)
```